



Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness



COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Selecting and Planning the Software Development

Methodology for Your Own Project

Software Development Methodologies

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 4
Output	Development methodology plan, selection worksheets, milestone map, online justification, and group presentation
Maximum Score	80 points

Student / Project Information

Block: A3-IBPH3	Date: September 18, 2026
System Title: RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	Instructor: Jean Y. Villasin, MIT
Members: RJ Cyrill Jirc L. Monzon, Mae D. Pelipas, Katrice B. Anunciado	

1. Activity Alignment and Purpose

Topic Learning Outcome: Compare common software development methodologies and select an appropriate approach for planning and managing the semester web-system project.

Course Outcome Alignment: Supports project planning and management by selecting and applying an appropriate development approach to the semester web-system project.

Syllabus Assessment: Development methodology plan; online methodology-selection justification.

Purpose of the Activity

Activities 2 and 3 established the architecture and technology stack of your semester project. In Activity 4, your team will decide how the project will be planned, developed, reviewed, and delivered. You will compare common methodologies, analyze project conditions, select a defensible primary approach, and map it to SDLC phases, iterations or sprints, and semester milestones. The final plan should become the team's working guide for managing development during the remaining laboratory activities.

Important distinction: SDLC phases describe the major categories of development work, while a methodology or development approach organizes how those phases are planned and repeated. Agile approaches do not remove requirements, design, testing, or deployment; they arrange work in adaptive cycles. DevOps and CI/CD are complementary delivery practices and do not replace the selected development methodology.

2. Intended Learning Objectives

- Explain the SDLC phases and the purpose of using a software development methodology.
- Distinguish predictive, iterative, incremental, adaptive, and hybrid development approaches.
- Compare Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, and Rapid Application Development (RAD).
- Compare Agile approaches including Scrum, Kanban, and Extreme Programming (XP).
- Explain how DevOps and CI/CD complement development and delivery without replacing the project methodology.
- Evaluate requirement stability, risk, scope, schedule, team capability, and stakeholder involvement when selecting a methodology.
- Map the selected methodology to SDLC phases, iterations, sprints, project milestones, reviews, and semester deliverables.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the comparison, selection, milestone planning, and oral defense.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Development Methodology Plan for the semester web-system project.

Activity Item	Requirement
Supporting Output	Methodology comparison matrix, scenario analysis, selection-criteria worksheet, SDLC mapping, semester milestone plan, DevOps/CI/CD delivery plan, 120-180 word methodology-selection justification, and group presentation.
Recommended Tools	Word/Google Docs, spreadsheet or timeline tool, diagrams.net/Draw.io, Canva or PowerPoint, GitHub Projects/Trello or an equivalent task board, and the assigned methodology case-study reading.
Presentation Format	5-7 minutes per group followed by 2-3 minutes of questions. Every member must explain an assigned selection criterion, project stage/iteration, or milestone.

4. Pre-Laboratory Requirements

- Approved system title and a brief statement of the community or organizational problem being addressed.
- Final or revised web-system architecture diagram from Laboratory Activity No. 2.
- Final or revised technology stack blueprint from Laboratory Activity No. 3.
- List of major system features, primary users, stakeholders, and any required approvals or feedback points.
- Known project constraints such as semester deadlines, technical risks, external services, uncertain requirements, and team limitations.
- The assigned methodology case-study reading and lecture notes on SDLC phases and software development methodologies.
- Access to the group GitHub repository and any current backlog, task board, calendar, or milestone list used by the team.

5. Part A - Development Approach Classification

Before comparing individual methodologies, classify the planning pattern correctly. Use the descriptions below to distinguish how a project responds to requirements, risk, delivery, and change. A specific methodology may combine more than one pattern.

Category	Meaning
SDLC phases	Planning, requirements/analysis, design, development, testing, deployment, and maintenance/review activities that every project must address in some form.
Predictive approach	Plans most scope, sequence, and major deliverables early; works best when requirements are relatively stable and change can be controlled.
Iterative approach	Repeats development cycles to refine the solution, requirements, design, or implementation using learning from earlier cycles.
Incremental approach	Builds and delivers the system in usable pieces or feature increments rather than waiting for one complete final release.
Adaptive / Agile approach	Uses frequent planning, feedback, and change response through short cycles or continuous flow.
Hybrid approach	Combines clearly defined practices from more than one approach because different parts of the project have different needs.
DevOps / CI/CD	Complementary delivery practices that support integration, testing, release, deployment, feedback, and operational reliability; they are not substitutes for the project methodology.

Methodology Comparison Worksheet

Methodology / Approach	Primary Planning Pattern	Strong Fit / Advantage	Limitation / Risk
Waterfall	Predictive; sequential	Stable, approved requirements	Late changes cause rework
V-Model	Predictive; paired test stages	Traceable verification	Rigid when requirements change
Prototyping	Iterative discovery	Clarifies screens with users	Prototype may hide design flaws
Iterative	Repeated refinement	Improves uncertain solutions	Rework can extend schedule
Incremental	Feature-by-feature delivery	Early usable functionality	Needs integration planning
Spiral	Iterative; risk-driven	Evaluates high-risk integrations	Risk analysis adds overhead
Rapid Application Development (RAD)	Adaptive; rapid prototyping	Fast demo; reusable components	Needs users and skilled team
Scrum	Adaptive; iterative/incremental	Time-boxed goals and reviews	Needs active backlog ownership
Kanban	Adaptive; continuous flow	Visible work; limits work in progress	No fixed review cadence by default
Extreme Programming (XP)	Adaptive; short engineering cycles	Pairing, tests and refactoring	Needs engineering discipline

6. Part B - Project Methodology Selection Criteria

Evaluate the conditions of your own project. Record evidence in the second column, then use the comparison cues to decide whether the condition favors a more predictive or a more iterative/adaptive approach. Your final choice should be based on the combined evidence, not on one criterion alone.

Selection Criterion	Your Project Evidence / Condition	More Predictive Tends to Fit When...	More Iterative / Adaptive Tends to Fit When...	Team Interpretation
Requirement stability	Pothole scope is fixed; UI and sync details may change.	Requirements are known and unlikely to change.	Requirements are expected to evolve through feedback or discovery.	Adaptive refinement within fixed scope.
Technical / project risk	ONNX inference, GPS and offline retry need device tests.	Technology and integrations are familiar with few unknowns.	High uncertainty, new technology, risky integrations, or research-heavy work require learning cycles.	Test risky paths early in sprints.
Scope clarity	Mobile capture, API, database and web map are defined.	Major features and boundaries can be defined early.	Feature priorities or details may change as prototypes and feedback are reviewed.	Deliver small integrated increments.
Schedule / deadline	Semester deadline; exact dates not supplied.	Formal phase gates and fixed planned deliverables are practical.	Frequent usable increments or short time-boxes are needed before the final deadline.	Use two-week sprints and reviews.
Team size and experience	Student group shares mobile, API and web tasks.	Roles can be specialized and work can follow a planned sequence.	A small cross-functional team can collaborate frequently and adjust tasks quickly.	Share work; peer review each change.
Stakeholder involvement	Instructor reviews; driver/dashboard feedback planned.	Stakeholders are available mainly at defined approval points.	Stakeholders can provide frequent feedback, review prototypes, or reprioritize work.	Schedule a demo at each sprint end.
Need for change and feedback	Field tests may reveal duplicate records or missed saves.	Changes are rare, costly, or tightly controlled.	Learning, testing, and user feedback are expected to change the next iteration or work queue.	Reprioritize defects in the backlog.

Selection rule

Choose a methodology because it fits the project conditions, not because it is popular or familiar. A hybrid approach is acceptable only when the team identifies which practices are combined and where each one applies. DevOps and CI/CD may complement any selected methodology but should not be presented as the methodology itself.

7. Part C - Methodology Scenario-Analysis Workshop

For each scenario, identify a defensible methodology or approach and explain the tradeoff. More than one answer may be acceptable when the reasoning is supported by the project conditions.

Scenario	Key Project Condition	Best-Fit Methodology	Why It Fits	Main Risk / Limitation	Alternative / Hybrid Option
1. Government registration module	Requirements are defined by fixed forms, rules, and approval steps.	Waterfall	Fixed forms allow phase approval.	Policy changes cause rework.	V-Model if test traceability is vital.
2. New citizen-reporting interface	Users are unsure what screen flow will be easiest until they see a working prototype.	Prototyping	Users can try and refine screen flow.	May neglect architecture.	Scrum with UI prototypes.
3. Verification-intensive transaction module	Each development stage requires a corresponding test or verification activity.	V-Model	Pairs requirements and design with tests.	Late changes are costly.	Iterative delivery with test gates.
4. Semester project with frequent adviser feedback	A small team must show working features regularly and reprioritize based on reviews.	Scrum	Regular increments and review feedback.	Needs available reviewers.	Kanban with scheduled demos.
5. Short deadline for an early working demo	The team needs rapid construction, reusable components, and strong stakeholder availability.	RAD	Reuse and quick user feedback speed a demo.	Speed can reduce quality.	Prototyping then incremental build.
6. GIS/ML feature with uncertain integration risk	External APIs, data quality, or model behavior may create technical uncertainty that must be evaluated repeatedly.	Spiral	Risk analysis and experiments repeat.	Heavy for a small team.	Scrum with early technical experiments.

Scenario-analysis checklist:

- Use evidence from requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Do not select a methodology only because the team already knows it or because it is currently popular.
- Distinguish the development methodology from tools such as Trello/GitHub Projects and from complementary DevOps/CI/CD practices.
- State at least one tradeoff, limitation, or alternative approach for every scenario.

8. Part D - Select and Map Your Project Methodology

Selected primary methodology:

Scrum - adaptive, iterative and incremental

Complementary approach / practice(s): UI prototyping; Git reviews; CI and controlled releases

Why this choice fits the project (3-5 sentences):

RoadWatch needs repeated device tests for detection, GPS and offline saving. Scrum organizes this work into small, reviewable increments. Two-week sprints let the team refine the mobile and web experience from feedback. Early integration checks reduce the risk of a failed final demonstration.

Project Methodology and SDLC Mapping Worksheet

Map the major SDLC activities to the way your selected methodology will actually organize the semester project. If the methodology repeats phases, show where the repetition occurs. If it uses sprints or continuous flow, identify the planning and review points clearly.

SDLC Phase / Activity	How It Is Handled	Time-Box / Duration	Iteration / Sprint / Stage	Main Team Role(s)	Stakeholder Check	Deliverable / Exit Evidence
Project planning	Set scope; rank risks	Week 1	Preparation	Whole team; PO	Instructor scope review	Backlog; risk list
Requirements / analysis	Refine stories and criteria	Each sprint start	Sprints 1-4	PO; developers	Review user needs	Accepted stories
Design	Refine UI, API and data	Within each sprint	Sprints 1-4	Mobile/API/web leads	Prototype feedback	Updated designs
Development / implementation	Build or refine features	2 weeks per sprint	Sprints 1-4	Developers	Sprint demo	Working increment
Testing / verification	Test before merging	Daily; sprint end	Every sprint	Developer + peer	Acceptance review	Test log; fixes
Integration / deployment	Integrate; release on LAN	Each sprint end	Sprints 1-4	API lead; reviewer	Approve stable build	Tagged release
Review / feedback	Demo; retrospective	End of each sprint	Weeks 3,5,7,9	Whole team; PO	Instructor/user review	Feedback; next tasks
Maintenance / improvement	Triage bugs; regression	Each sprint; W10	All; final buffer	Assigned developer	Confirm fixes	Closed issue; retest

SDLC mapping guidance

Waterfall and V-Model usually emphasize planned phases and formal transitions. Prototyping, Iterative, Incremental, Spiral, and RAD repeat or accelerate selected activities. Scrum normally organizes work into sprints; Kanban uses continuous flow and work-in-progress control; XP emphasizes short feedback cycles and engineering practices. Your mapping must reflect the actual method selected by the team.

9. Part E - Semester Project Milestone Plan

Translate the selected methodology into a realistic semester schedule. Every milestone should have a measurable output and a review, approval, demonstration, test, or feedback point. Use your actual course deadlines when they are already known.

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Week 1	Preparation	Current baseline; pothole scope	Audit functions; rank risks	Backlog; acceptance criteria	Instructor scope check
Weeks 2-3	Sprint 1	Detection and GPS reliability	Test existing ONNX and camera	Saved pothole with GPS/time	Demo + retrospective
Weeks 4-5	Sprint 2	Offline save and retry	Test LIVE and manual save	Queue survives offline use	Reconnect demo + review

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Weeks 6-7	Sprint 3	Web map and statistics	Check API/DB/web consistency	Synced record visible on map	Dashboard user review
Weeks 8-9	Sprint 4	Integrated release	Regression; fix major defects	Tagged LAN release; test log	End-to-end demo + review
Week 10	Final buffer	Documentation; handover	Retest; rehearse defense	Final PDF; release notes	Instructor evaluation
Schedule note	Proposed relative weeks	Apply to remaining course work	Align with official deadlines	No completed tests claimed	Team confirms dates

Milestone planning rule

The methodology plan must be visible in the schedule. A sprint-based plan should show sprint goals and reviews; an incremental plan should show usable feature increments; a predictive plan should show phase gates and approval points; a Kanban-style plan should show how work enters, moves through, and exits the workflow. Do not use vague milestones such as “continue coding” without an observable deliverable.

10. Part F - DevOps and CI/CD Complementary Delivery Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Version-control workflow	How branches, commits, merges, and code reviews support the selected methodology.	Feature branches; peer-reviewed pull requests into main.
Continuous integration	When code is integrated and what build/check should run before changes are accepted.	Plan CI on each PR: Flutter analyze/test and API tests.
Testing practice	What unit, API, database, UI, or acceptance checks will be performed and when.	Test save, GPS, API validation, retry and dashboard each sprint.
Database change control	How PostgreSQL schema changes, migrations, and seed data will be coordinated with application development.	Version SQL migrations; back up data; test on a separate database.
Deployment stages	Local, test/staging, and final deployment targets; identify any approval before release.	Local development -> LAN test -> approved LAN release.
Environment configuration	How API URLs, credentials, and other environment-specific values are kept outside source code.	Keep DB secrets in ignored .env; configure mobile API URL per build.
Monitoring / feedback	How errors, defects, stakeholder feedback, and completed work are recorded and fed into the next cycle or milestone.	Log defects with steps and screenshots; prioritize at sprint review.
Release / rollback	How the team will identify a stable release and recover from a failed deployment or defective change.	Tag stable builds; retain prior build and DB backup for recovery.

Recommended minimum methodology evidence in the project repository or shared workspace (adapt as needed):

```
docs/methodology/
  development-methodology-plan.pdf
  methodology-selection-justification.md
```

```

milestone-plan/
sprint-or-iteration-notes/
review-and-feedback-records/
test-evidence/
release-notes/
README.md - brief description of the team's development workflow

```

11. Part G - Construct the Development Methodology Plan

Create one clear project-specific methodology plan that explains how your team will manage the semester project from requirements through delivery. The plan may be presented as a structured one-page or multi-page visual, timeline, workflow, or combination of diagram and schedule, but it must remain consistent with the completed worksheets.

No.	Required Element	What Must Be Shown
1	Project identity	Official system title, project problem, major users/stakeholders, and team members.
2	Selected primary methodology	Exact methodology/approach selected and whether it is predictive, iterative, incremental, adaptive, or hybrid.
3	Selection rationale	Project evidence from requirement stability, risk, scope, schedule, team, and stakeholder involvement.
4	SDLC mapping	How planning, requirements, design, development, testing, deployment, and review are organized.
5	Iteration / sprint / flow structure	Length or sequence of iterations/sprints/stages, or the workflow used if the method does not use fixed sprints.
6	Requirements / backlog control	How features, tasks, priorities, changes, and acceptance criteria will be recorded and updated.
7	Team responsibilities	Who plans, develops, tests, reviews, documents, coordinates, or approves work.
8	Semester milestones	Major target dates or weeks, expected deliverables, demonstrations, and completion criteria.
9	Stakeholder involvement	When users, instructor, adviser, client, or other stakeholders provide feedback or approval.
10	Quality and testing	When testing occurs and how defects, rework, verification, or acceptance are handled.
11	DevOps / CI/CD support	Git/GitHub workflow, integration, build/test, database change control, deployment, and feedback practices.
12	Adaptation / control rule	How the team responds when requirements, risks, schedule, or technical conditions change.

Minimum Plan Rule

Your plan must show a complete management path from project requirements or backlog -> planned phase/iteration/sprint/work queue -> development and testing -> review/feedback -> milestone or release. If the selected method is iterative or adaptive, show how the cycle repeats. If it is predictive, show phase gates. If it is hybrid, label exactly where each approach is used.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the decision-and-planning logic expected in the final output. Replace the generic labels with the actual constraints, methodology, iteration/stage structure, milestones, stakeholder reviews, and delivery practices of your own project.

RoadWatch project-specific Part G plan: see attached page after the original template.

Reference Methodology Selection and Semester Planning Flow

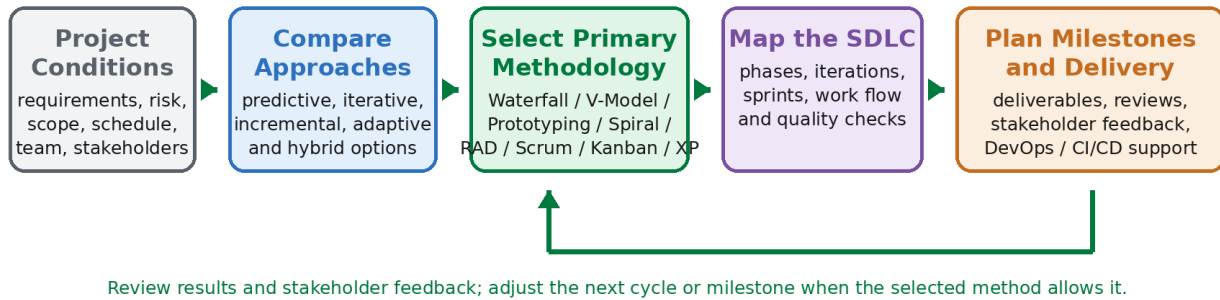


Figure 1. Reference methodology selection and semester planning flow.

Step	Reference Flow
1	Identify project conditions: requirement stability, risk, scope, schedule, team, and stakeholder involvement.
2	Compare predictive, iterative, incremental, adaptive, and hybrid options using the actual project evidence.
3	Select one primary methodology and identify any complementary practices such as prototyping, DevOps, or CI/CD.
4	Map SDLC phases to stages, iterations, sprints, or continuous workflow according to the selected methodology.
5	Define semester milestones, deliverables, testing points, demonstrations, reviews, and stakeholder feedback.
6	Use results and feedback to continue the planned next stage or adapt the next cycle when the selected methodology permits it.

13. Part H - Methodology-to-Project Trace

Selected feature / work package:

Offline pothole saving and reconnection synchronization

Target milestone / release: Sprint 2 review - end of Week 5

Step / Iteration	Planned Work	Methodology Principle / Practice	Evidence / Deliverable	Review / Feedback
Backlog	Define offline save	User story; clear criteria	Story: retain and sync record	PO reviews scope
Sprint 1 check	Inspect existing queue	Early risk discovery	LIVE/manual save gap list	Team reviews risks
Sprint 2 plan	Select retry tasks	Sprint goal; capacity	Tasks and acceptance checks	Team commits goal
Design	Define saved fields	Small design increment	ID, GPS, time, sync status	API/mobile review
Implement	Refine queue + retry	Integrated increment	Updated mobile/API code	Peer code review
Offline test	Save; restart app	Continuous verification	Pending record still available	Tester checks log
Reconnect test	Retry same record	Acceptance testing	One stored record; no duplicate	Inspect DB + map
Sprint review	Demo complete flow	Inspect and adapt	Demo notes; accepted story	User/instructor input
Retrospective	Plan remaining fixes	Improve next sprint	Sprint 3 backlog updates	Team selects actions

14. Part I - Methodology-Selection Justification

Write 120-180 words explaining why the selected development methodology is appropriate for your own semester project. This will also serve as the basis of the online methodology-selection justification. Address all of the following:

- Name the selected primary methodology and classify it as predictive, iterative, incremental, adaptive, or hybrid as appropriate.
- Use project evidence related to requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Explain why the selected methodology is more appropriate than at least two reasonable alternatives.
- Describe how SDLC phases, iterations, sprints, work flow, and semester milestones will be organized.
- Explain how DevOps and CI/CD practices will complement development and delivery, including one limitation or risk the team must manage.

We selected Scrum for RoadWatch because it supports adaptive, iterative and incremental development. The pothole-focused scope is defined, but camera behavior, GPS accuracy, offline saving and synchronization require repeated device testing. Our student team must coordinate Flutter, FastAPI, PostgreSQL and the web dashboard while meeting a semester deadline. Planned instructor reviews and user demonstrations will help refine priorities. Waterfall would delay feedback on these integrations, while Spiral would add more formal risk-analysis work than our semester plan needs. We propose four two-week sprints after an initial planning week, followed by a final review buffer. Each sprint includes requirements refinement, design, implementation, testing, a working demonstration and a retrospective. Milestones cover detection and GPS, offline synchronization, dashboard consistency and an integrated release. Git branches, peer review, automated checks and controlled deployment will support delivery. Automated tests cannot fully reproduce camera, GPS or network conditions, so physical-device testing remains necessary. Feedback enters the backlog; lower-priority work is deferred to protect the sprint goal and final deadline.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The official project title, users, features, and real project conditions are used.	☐ The selected primary methodology is clearly named and correctly classified.
☐ Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP were compared.	☐ Selection criteria use actual evidence about requirements, risk, scope, schedule, team, and stakeholders.
☐ Scenario analysis includes a methodology choice, reasoning, tradeoff, and alternative.	☐ The team explains why at least two reasonable alternative methodologies were not selected.
☐ SDLC phases are mapped consistently to the selected methodology.	☐ Iterations, sprints, stages, or continuous workflow are represented accurately.
☐ The semester milestone plan includes dates/weeks, deliverables, and review or feedback points.	☐ Stakeholder involvement and change-management points are visible in the plan.
☐ DevOps and CI/CD are treated as complementary delivery practices, not as the primary methodology.	☐ The plan is feasible for the team size, available time, technical risks, and semester deadline.
☐ The written methodology-selection justification agrees with all worksheets and the final plan.	☐ Every group member can explain the methodology, milestone sequence, and major tradeoffs without reading a script.

16. Submission Requirements

- PDF of the final Development Methodology Plan in a clear, readable format.
- Completed Parts A-H and the 120-180 word methodology-selection justification.
- Export or screenshot of the semester milestone timeline, sprint/iteration plan, or task board used to support the final plan.
- Updated project GitHub repository or shared project workspace containing the methodology artifacts under a folder such as docs/methodology/.

- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA4_SystemName.
- Deliver a 5-7 minute group presentation using the final methodology plan as the main visual, followed by 2-3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own project conditions and planning decisions. AI tools may be used to clarify methodology concepts or review ideas, but the final comparison, selection criteria, methodology choice, SDLC mapping, milestone plan, and written justification must be checked, understood, and defended by the team. The instructor may ask any member to explain why a methodology was selected or trace how one feature will move through the planned process without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unclear reasoning, unequal participation, excessive reading, inconsistent milestone mapping, or inability to defend the selected methodology against reasonable alternatives.

Criterion	Evidence for Full Credit	Points
Methodology comparison accuracy	The comparison correctly distinguishes the major characteristics, strengths, limitations, and appropriate use of Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP.	10
Scenario analysis and tradeoff reasoning	Each scenario uses relevant project conditions to select a defensible method, explains why it fits, identifies a limitation, and considers a reasonable alternative or hybrid option.	8
Project-specific selection criteria	Requirement stability, risk, scope, schedule, team capability, stakeholder involvement, and need for change/feedback are evaluated using evidence from the actual project.	10
Selected methodology and written justification	The primary methodology is clearly named, correctly classified, supported by project evidence, compared with alternatives, and justified in a coherent 120-180 word explanation.	12
SDLC, iteration, sprint, or flow mapping	Planning, requirements, design, development, testing, deployment, review, and improvement are mapped accurately to the selected method and its stages, cycles, sprints, or continuous workflow.	12
Semester milestone plan	The schedule is realistic, project-specific, and aligned with the selected methodology; milestones contain measurable scope, deliverables, review/feedback points, and completion evidence.	12
DevOps and CI/CD complementary delivery plan	Version control, integration, testing, database change control, deployment, configuration, monitoring/feedback, and release/rollback practices appropriately support the selected methodology.	6
Group presentation and oral defense	The presentation is clear, organized, and within time; all members participate and correctly defend the methodology choice, project mapping, milestones, tradeoffs, and delivery practices.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which project selection criterion had the greatest influence on your methodology choice, and why?

Technical risk mattered most: camera, GPS and offline retry must work together on a real phone. Short sprints let us discover and correct failures early.

Which reasonable alternative methodology did your group reject, and what project condition made it less suitable?

We rejected Waterfall because offline behavior and interface details need repeated feedback. A fixed sequence would make late integration changes harder.

Where will the most important iteration, sprint review, phase gate, or workflow review occur in your semester plan?

The key review is Sprint 2 at Week 5: save a pothole offline, restart the app, reconnect, and verify one matching record on the dashboard.

How will stakeholder feedback or requirement changes be handled without losing control of scope and schedule?

The product owner records changes in the backlog with priority and acceptance criteria. The team negotiates scope and defers lower-priority tasks to protect deadlines.

Which DevOps or CI/CD practice should your team establish first, and what evidence will show that it is working?

Establish peer-reviewed Git branches first. Merged pull requests, passing checks and a tagged working build will demonstrate that the workflow is operating.

Part G - RoadWatch Development Methodology Plan

Project identity and scope

RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System. Drivers capture potholes; dashboard users review locations and statistics. The current delivery focuses on potholes. Preserve the existing ONNX model, Flutter camera/GPS, FastAPI, PostgreSQL and web map. Team members are listed on the cover.

Primary method and planning basis

Scrum: adaptive, iterative and incremental. This is a proposed remaining-work plan, not a claim that all tests or practices already exist. Use Week 1 for baseline review, four two-week sprints for refinement and integration, and Week 10 for handover. Confirm dates and available team capacity against the course calendar.

Management path and repeating cycle

Prioritized backlog -> sprint planning and goal -> requirements/design -> development and testing -> working increment -> sprint review and retrospective. Review feedback returns to the backlog for the next sprint. Hold a 15-minute Daily Scrum on working days; record blockers and adjust the daily plan.

Responsibilities and stakeholder participation

Assign one member as product owner to order the backlog and clarify acceptance criteria, one as Scrum Master to facilitate events, and developers across mobile, API/database, web, testing and documentation. Roles may overlap in the student team. Names remain for the group to assign. Invite the instructor and available driver/dashboard representatives to sprint-end reviews.

Requirements, quality and change control

Track each story with priority, owner, acceptance criteria and evidence. Board: Backlog, Ready, In Progress, Review/Test, Done. Done means peer-reviewed code, relevant passing checks, device verification where needed, updated notes and integrated functionality. Unfinished work returns to the backlog. Discuss new scope with the product owner; preserve the sprint goal and defer lower-priority additions.

Milestones and acceptance evidence

Week 1: agree scope and baseline. Weeks 2-3: verify detection with GPS/time. Weeks 4-5: offline save survives restart and syncs without duplicates. Weeks 6-7: synchronized data agrees across API, database, map and statistics. Weeks 8-9: regression checks and a tagged LAN release. Week 10: documentation, defect retests and group defense. The worksheets provide the milestone timeline.

Delivery practices and risk response

Plan CI for Flutter analysis/tests and API checks on pull requests; keep credentials outside Git. Version database changes, test against a separate database and retain backups. Promote local builds to LAN testing and release only after review. Tag known-good builds for rollback. Public hosting, HTTPS and access controls remain proposed future work. Device tests must cover camera, GPS, network loss and retry; CI alone cannot validate them.

Presentation and handover

Use the plan and filled milestone tables for a 5-7 minute presentation: project/rationale (1 minute), method and roles (1), sprint milestones (2), offline trace and tests (1), delivery/tradeoffs (1). Assign every member a section. Add the final plan, backlog/review notes, actual test evidence and release notes to docs/methodology/ before submission.

RoadWatch methodology plan attachment | Proposed schedule and practices; team validation required.